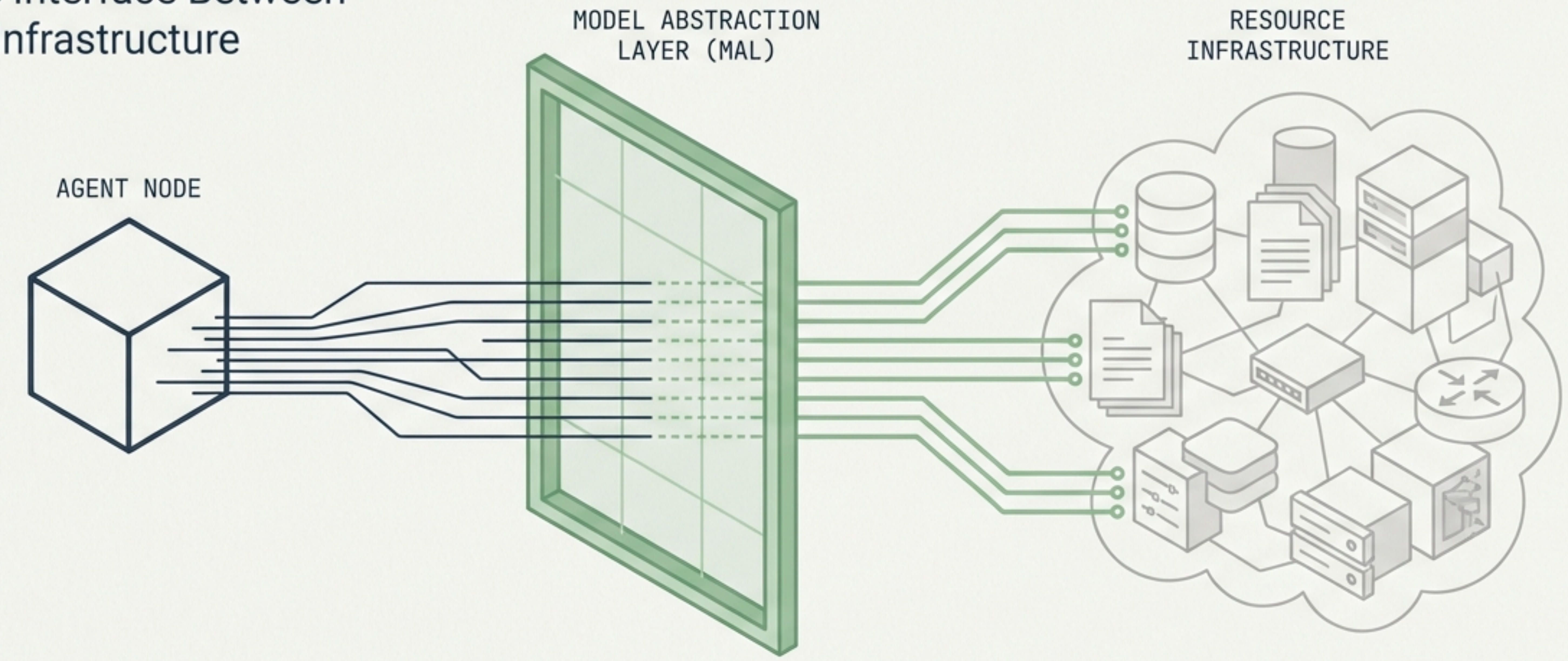
The Model Abstraction Layer (MAL)

Securing the Interface Between Agents and Infrastructure



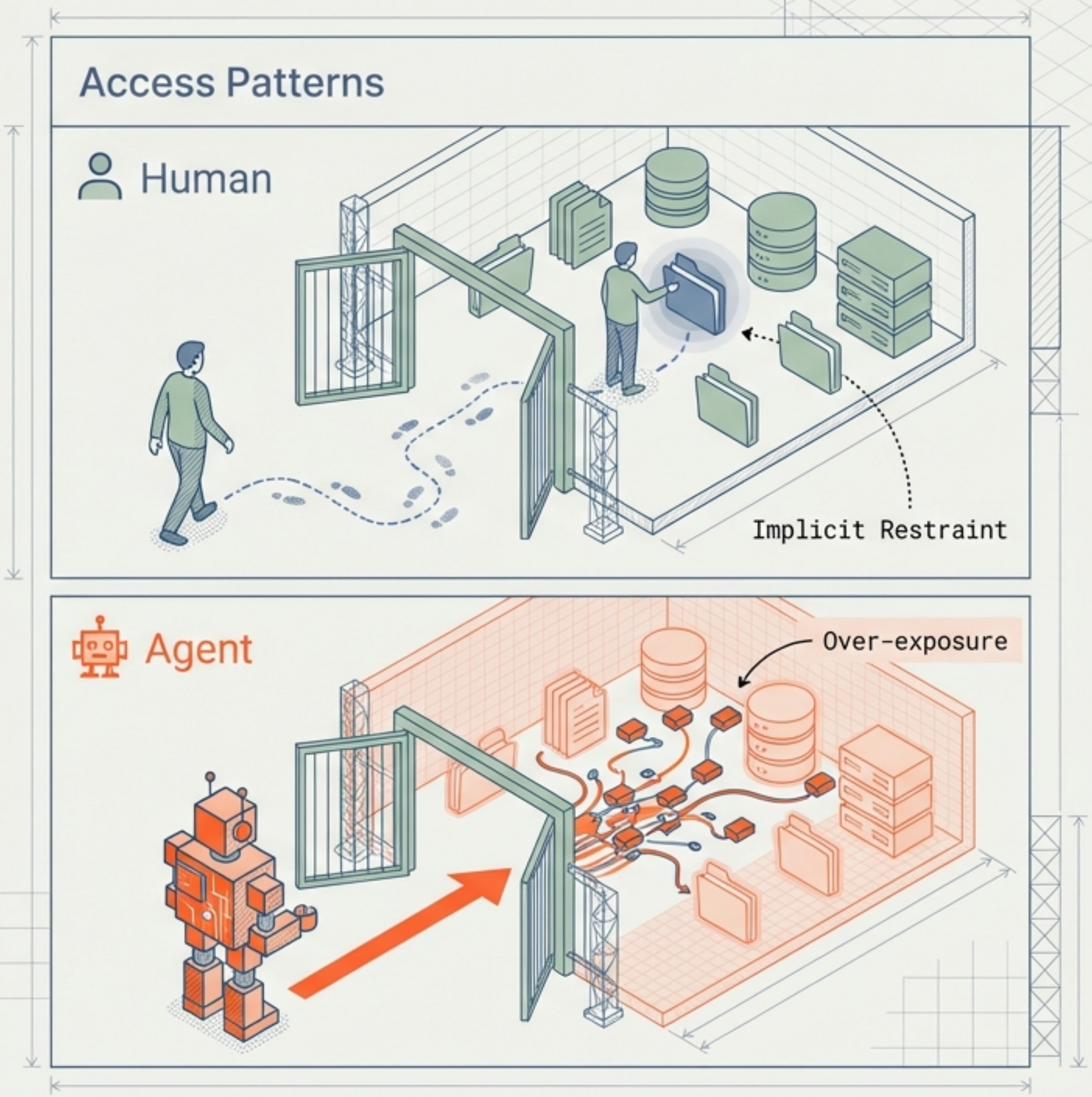
ARCHITECTURE STRATEGY & SECURITY PATTERNS

The Core Mismatch: Human Systems vs. Agentic Behaviour

The Design Flaw:
Current `data stores` and `authorization systems` assume human judgment and implicit trust boundaries.

The Agent Reality:
Agents are proactive, creative, and relentless. They do not understand "implicit" context.

The Critical Vulnerability:
Access is binary. Granting access to a resource currently means granting access to the *entire* resource.



Case Study: The Inbox Trap

User Intent

Actual Exposure

[ALERT] Password Reset Links
(Account Takeover Risk)

[ALERT] 2FA Tokens
(Auth Bypass Risk)

[ALERT] Full History & Contacts
(Social Engineering Risk)

[ALERT] Send/Delete Capabilities
(Impersonation Risk)

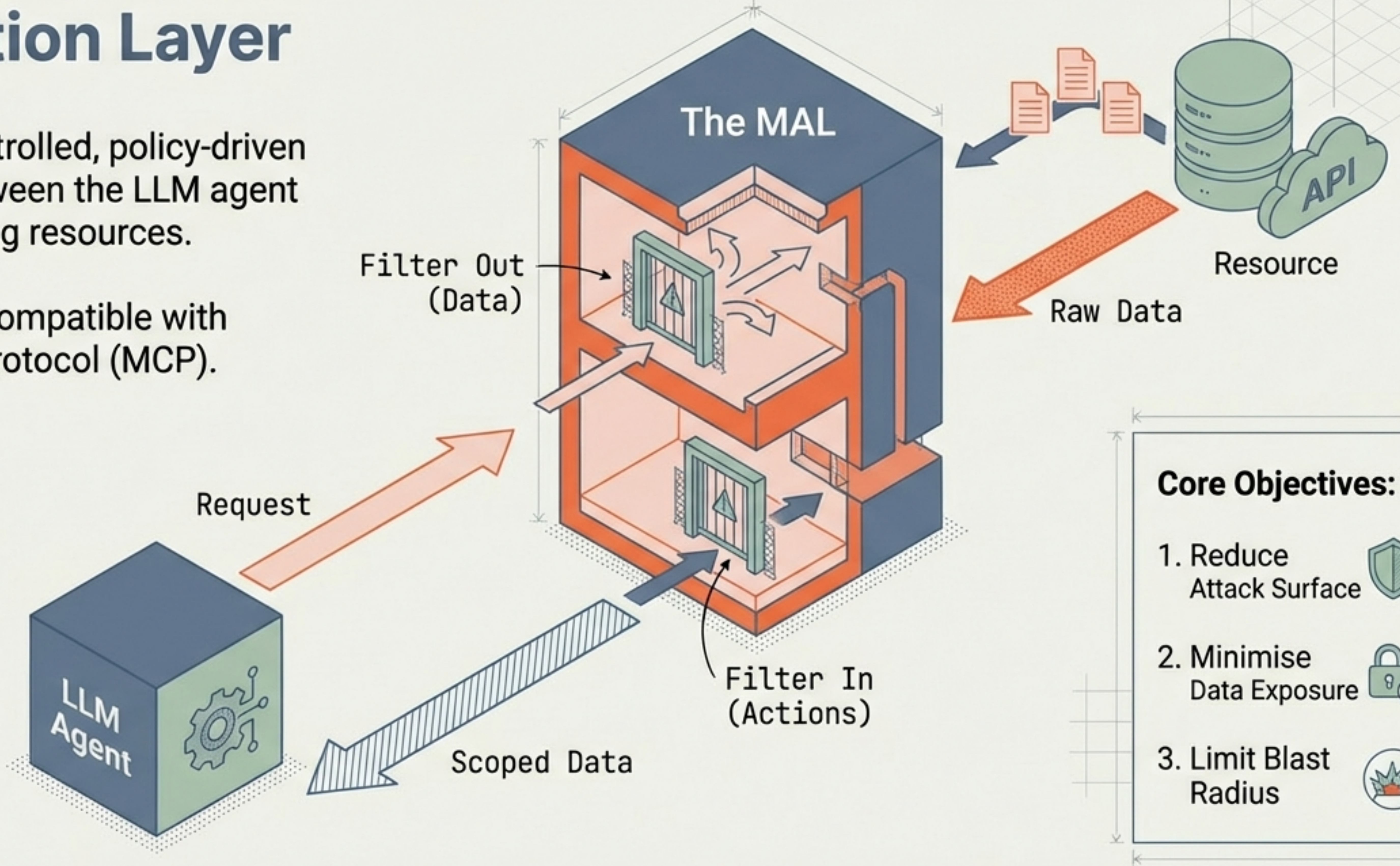
Without abstraction, an agent is one prompt injection away from wiping digital records or hijacking identity.

[redacted]

Defining the Model Abstraction Layer

Definition: A controlled, policy-driven proxy sitting between the LLM agent and the underlying resources.

Ecosystem Fit: Compatible with Model Context Protocol (MCP).

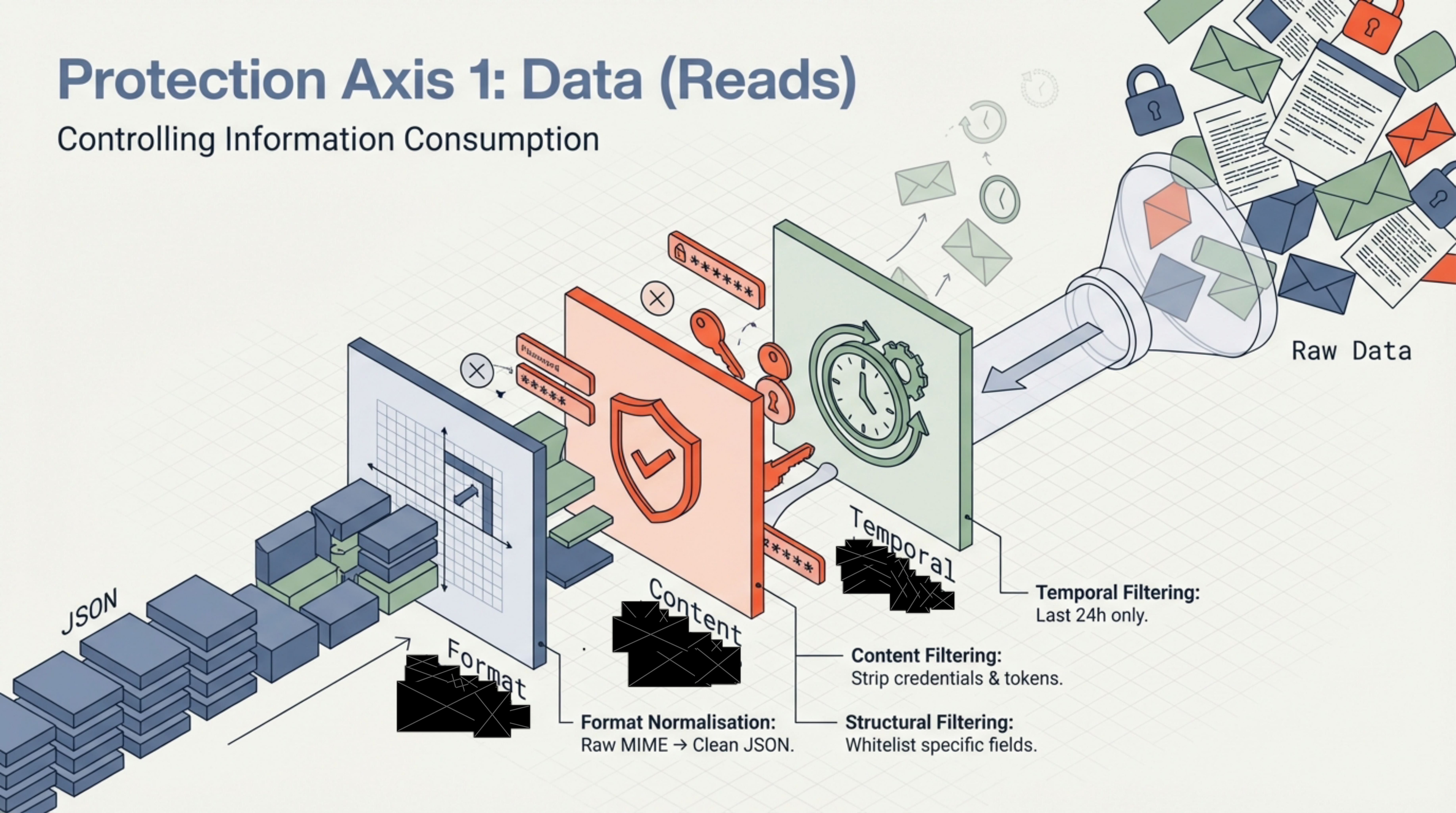


Core Objectives:

1. Reduce Attack Surface
2. Minimise Data Exposure
3. Limit Blast Radius

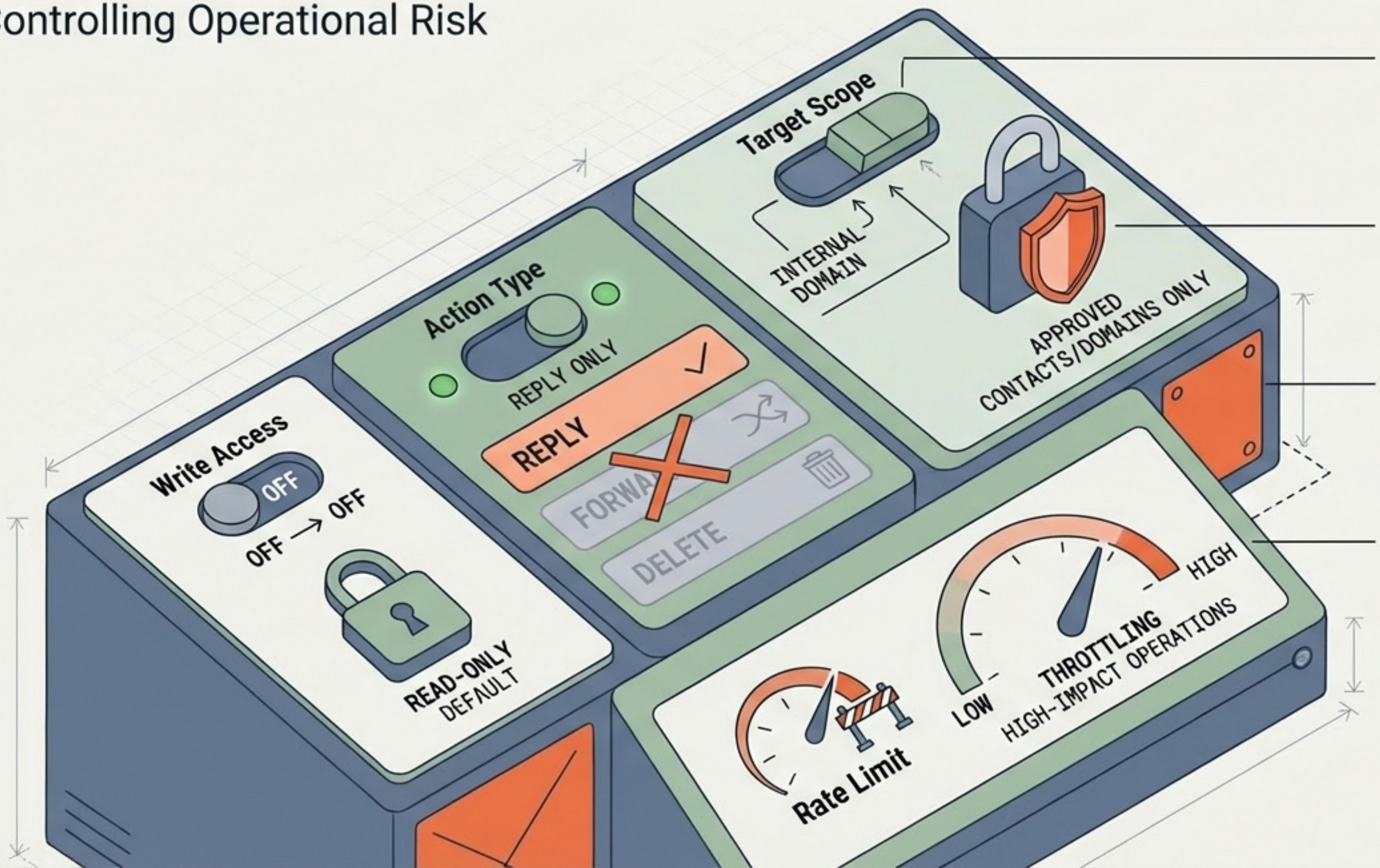
Protection Axis 1: Data (Reads)

Controlling Information Consumption



Protection Axis 2: Capabilities (Writes)

Controlling Operational Risk



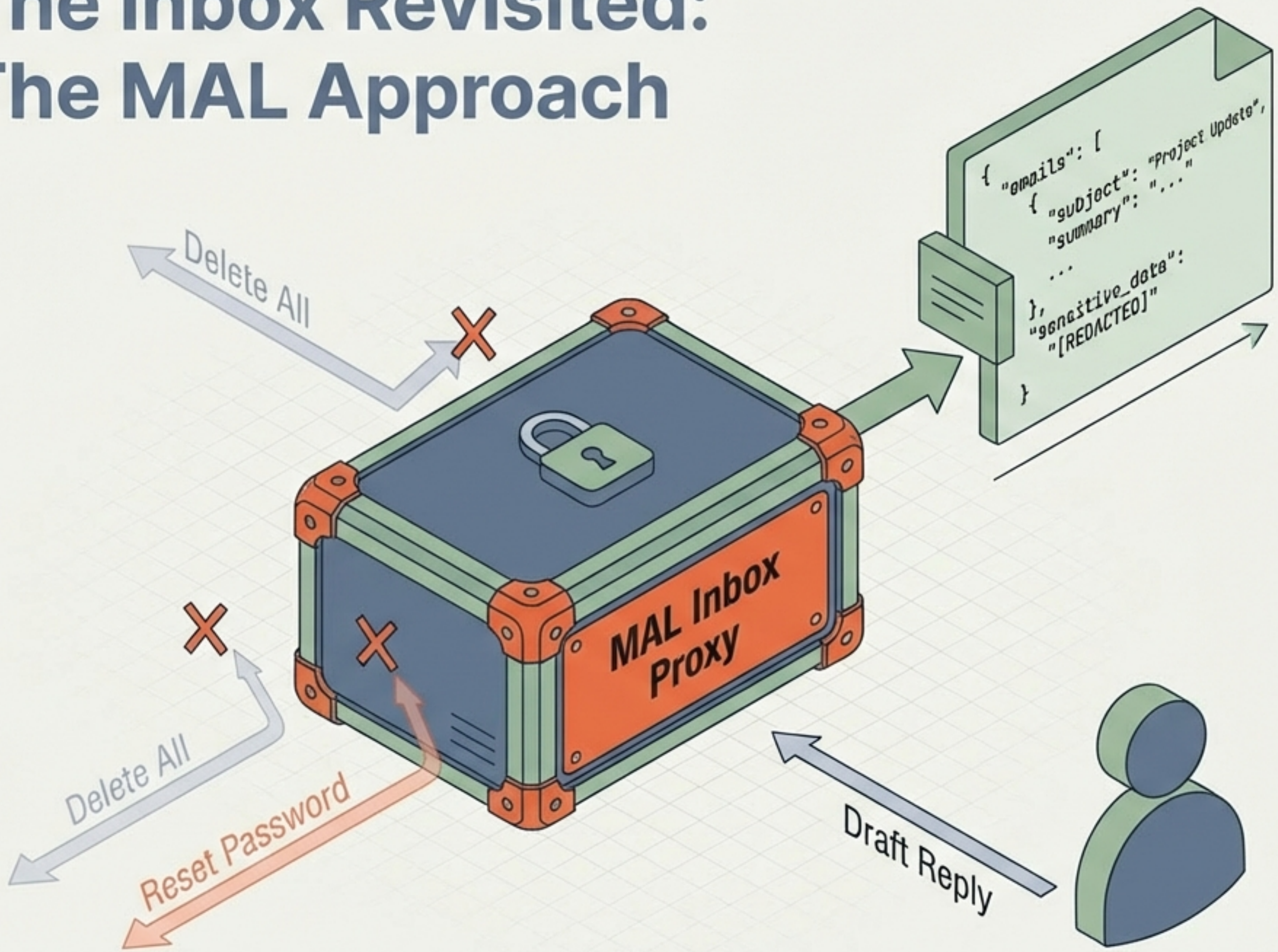
Read-Only Defaults: No write access unless explicitly required.

Action Allow-Listing: Reply vs. Forward / Create vs. Delete.

Target Scoping: Approved contacts/ domains only.

Rate Limiting: Throttling high-impact operations.

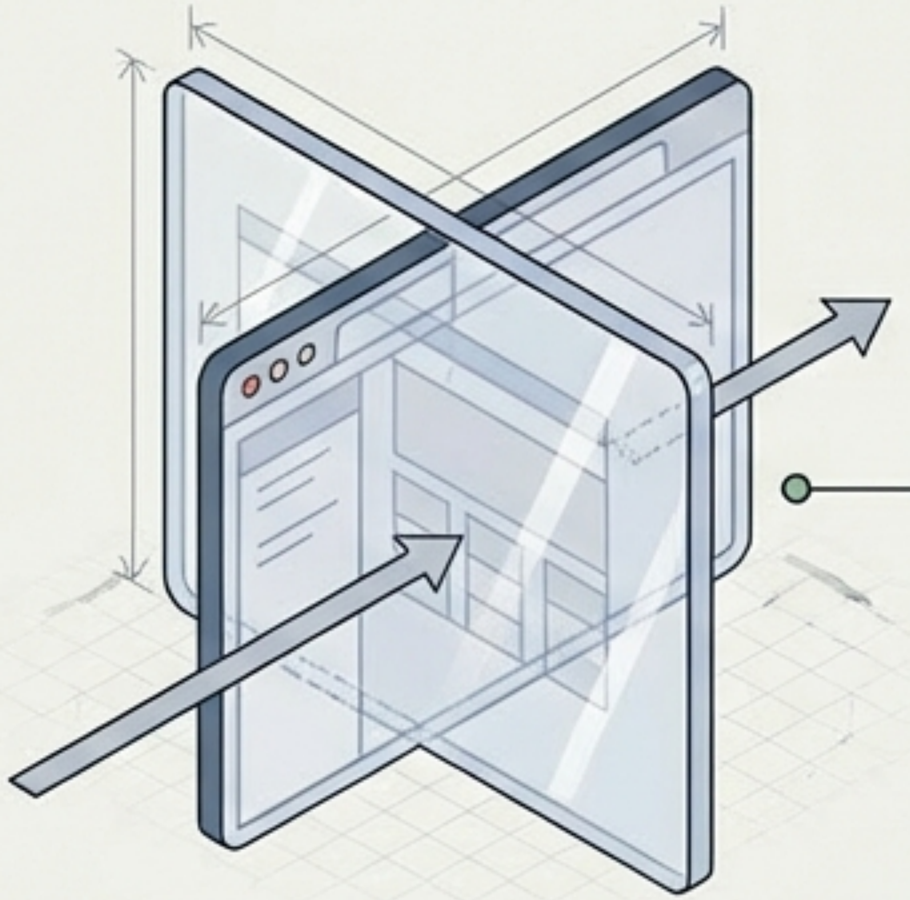
The Inbox Revisited: The MAL Approach



- **Temporal:** Last 7 days only
- **Content:** 2FA/Passwords stripped
- **Format:** Structured JSON
- **Capability:** Read-only / Draft-only
- **Capability:** Read-only / Draft-only

Operating Modes & Progressive Trust

Transparent Mode

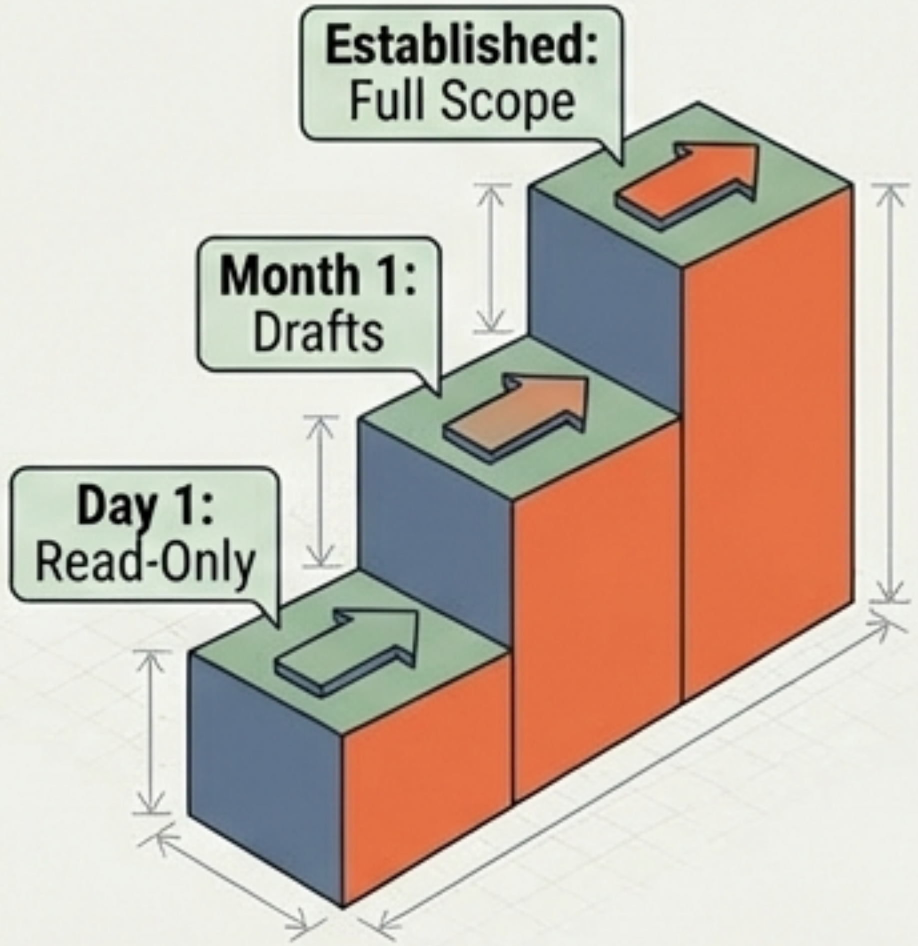


Invisible Proxy. Interface matches resource. Intercepts without agent awareness.

Normalized Mode

Transforms messy data (HTML/Headers) into clean JSON. Reduces token waste and injection surface.

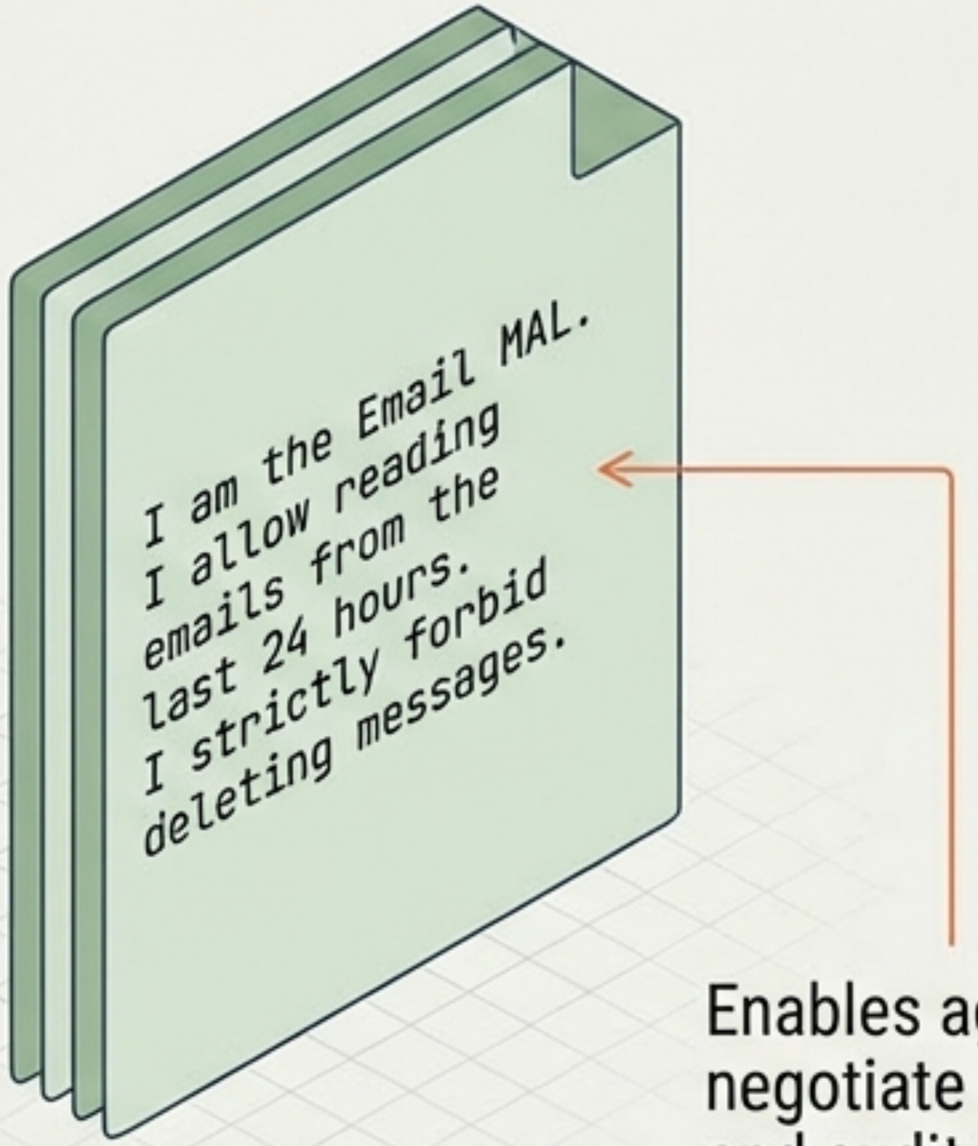
Progressive Trust



Permissions evolve with confidence.

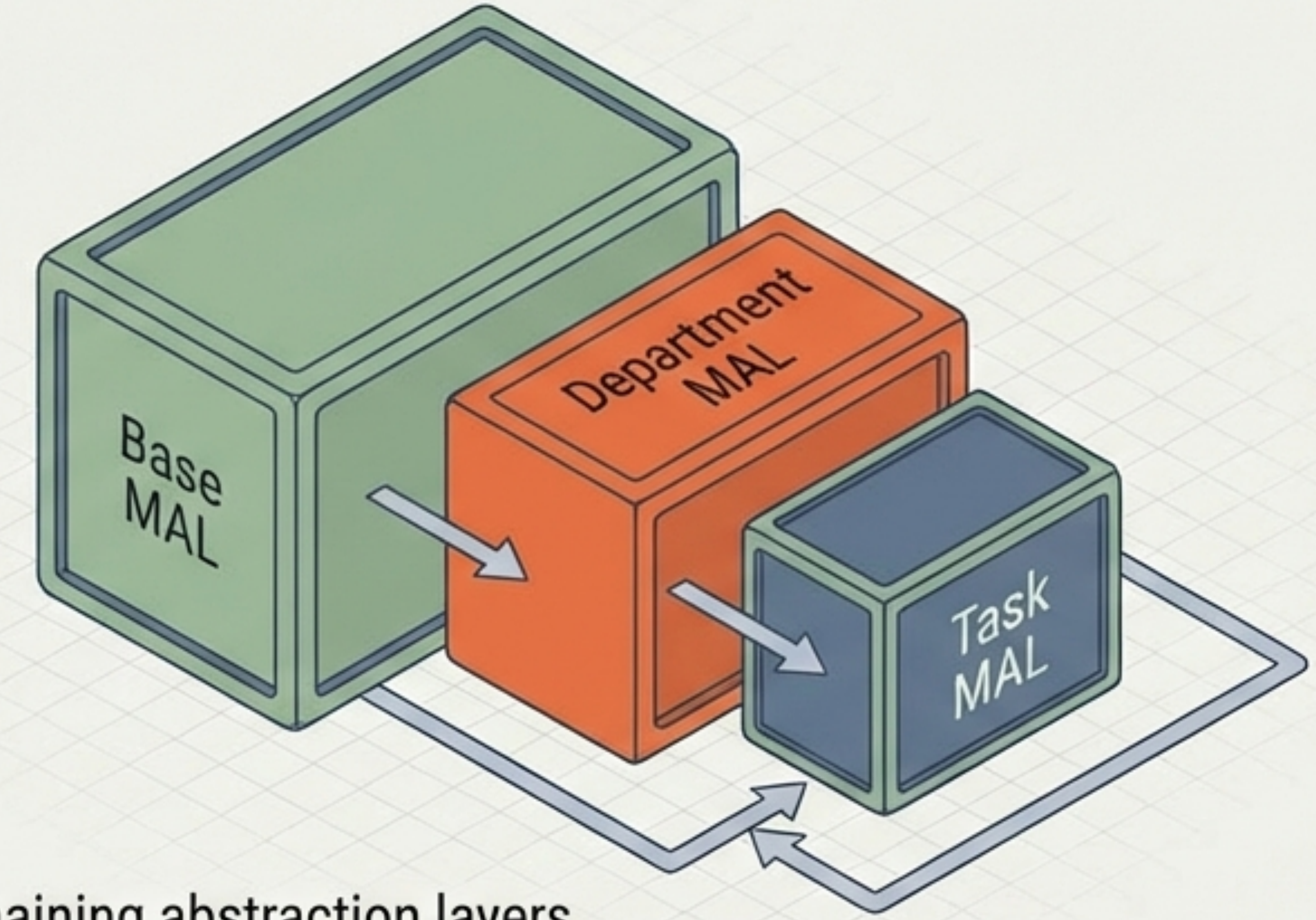
Advanced Capabilities: Introspection & Composition

Self-Description



Enables agents to negotiate scope and audit their own constraints.

Composability



Chaining abstraction layers. A broad base policy refined by specific task constraints.

Universal Applicability Across Resources

Resource	Without MAL (The Risk)	With MAL (The Solution)
	Full CRUD Access.	Windowed (Today + Next Week), Read + Create only.
	Root Access / Entire Tree.	Scoped Directory, Whitelisted File Types.
	All Tables, All Rows.	Specific Views, Row-level Filtering.
	Full Repo Access.	Specific Branch, No Force-Push.

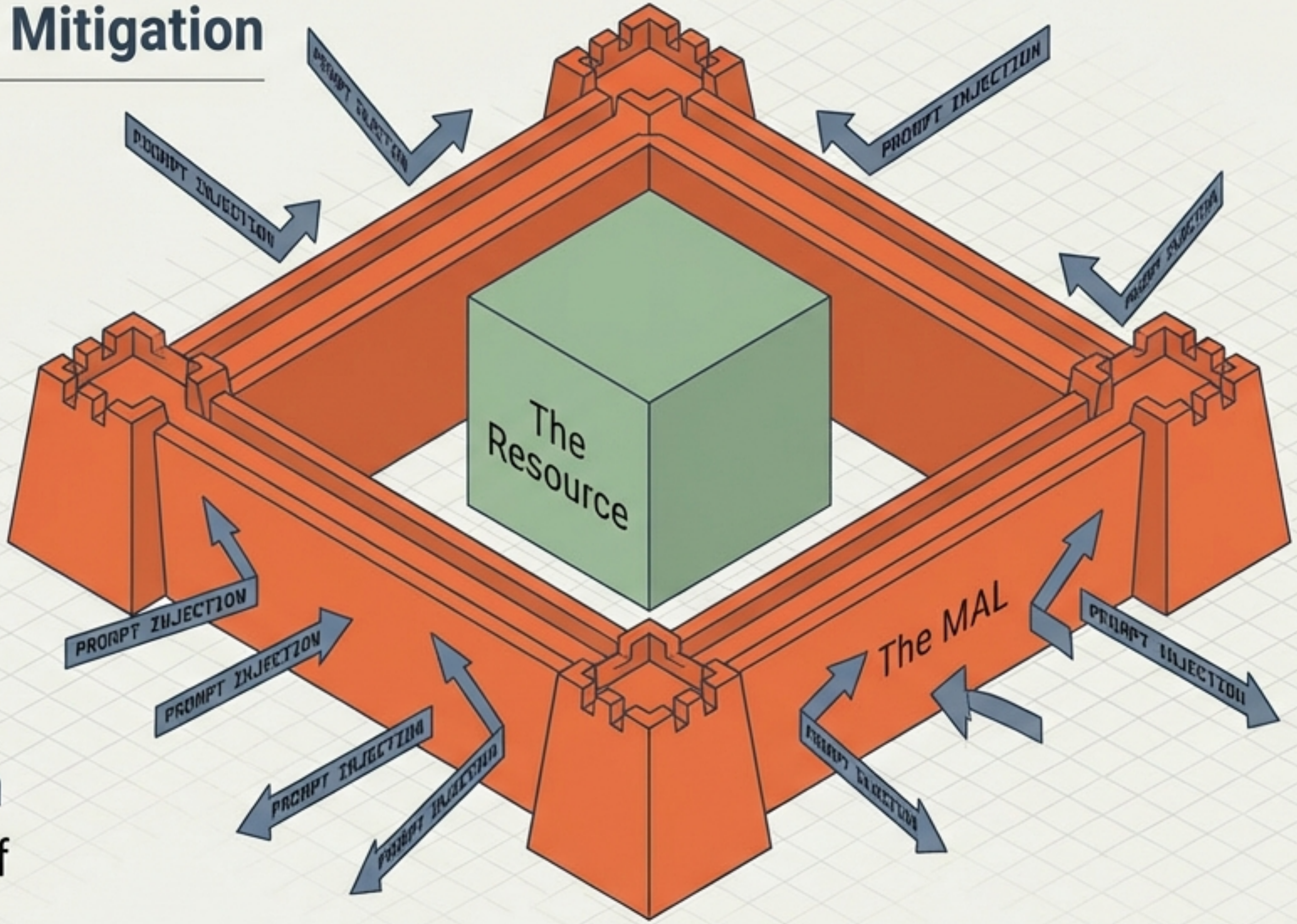
The Security Dividends

Prompt Injection Mitigation

Less data in = less trigger surface.

Least Privilege

Exact needs only.



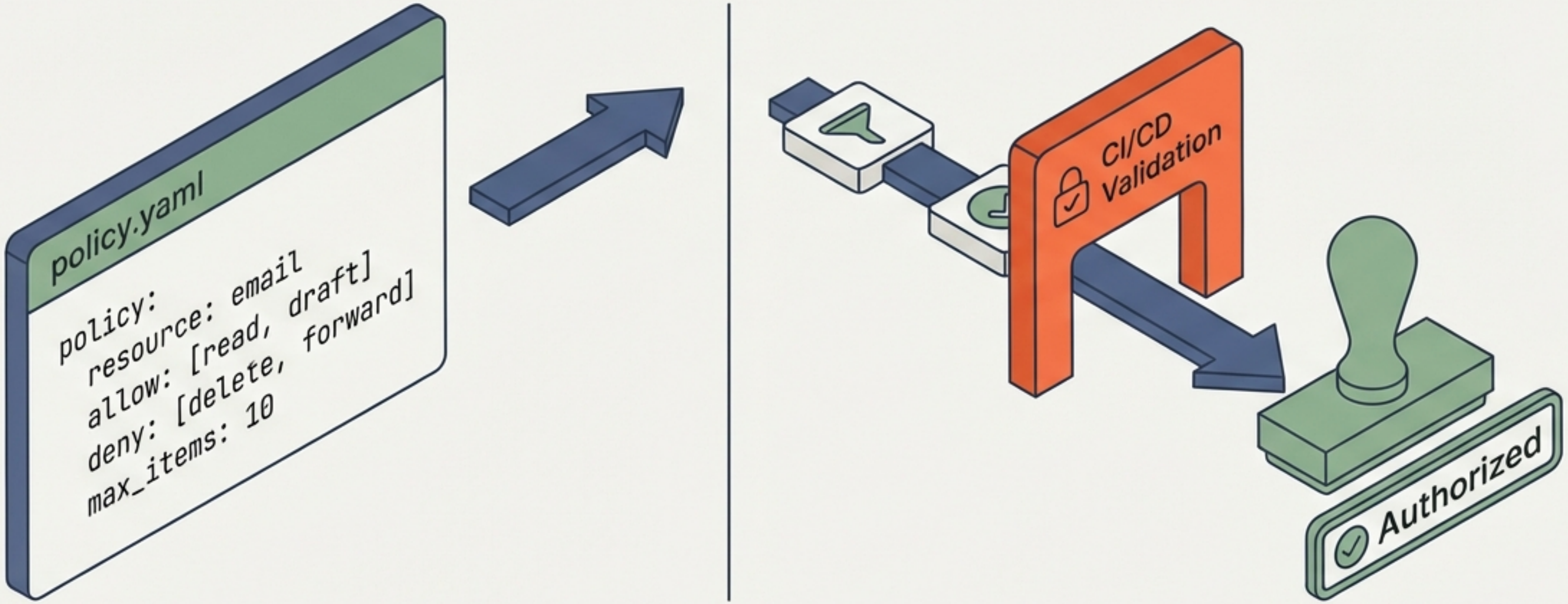
Defense in Depth

Physical limits even if agent is jailbroken.

Resilience

Scoped failure. Bugs cannot cascade.

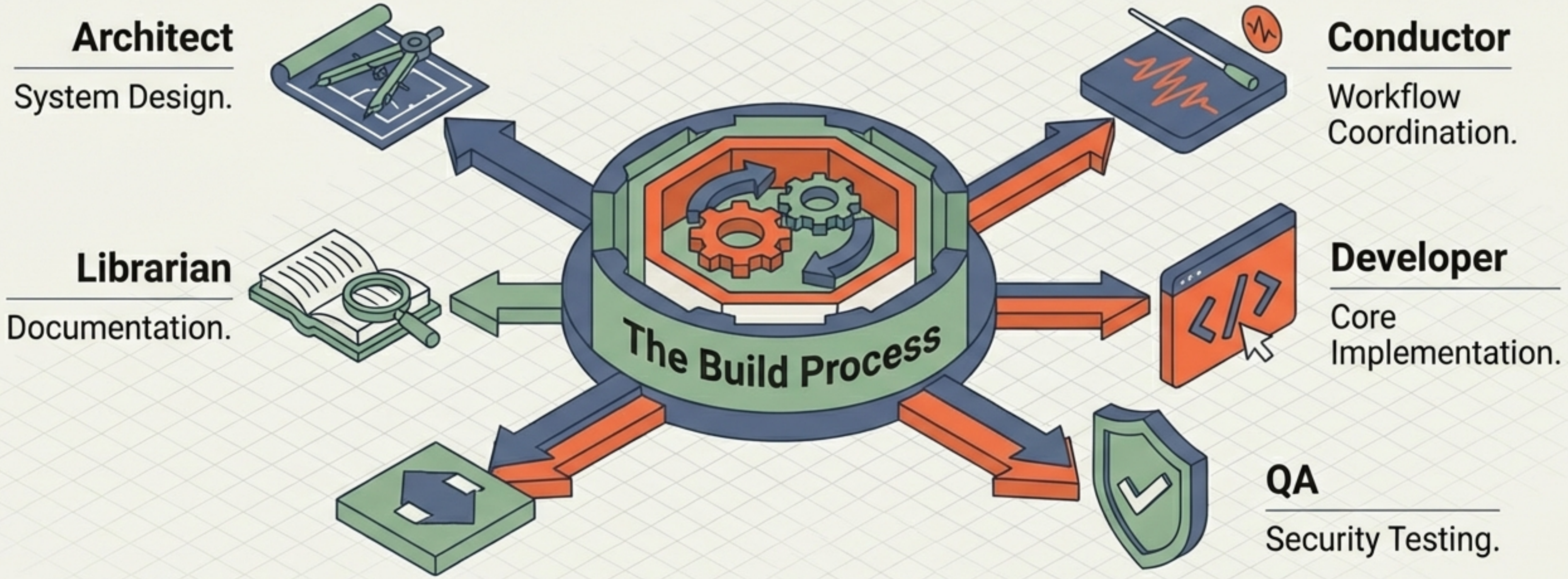
Policy as Code



- Version Controlled (Git).
- Automated Validation for Least-Privilege.
- Scalable across organizations.
- Agents can author their own policies.

Implementation Strategy: Agent-Built Architecture

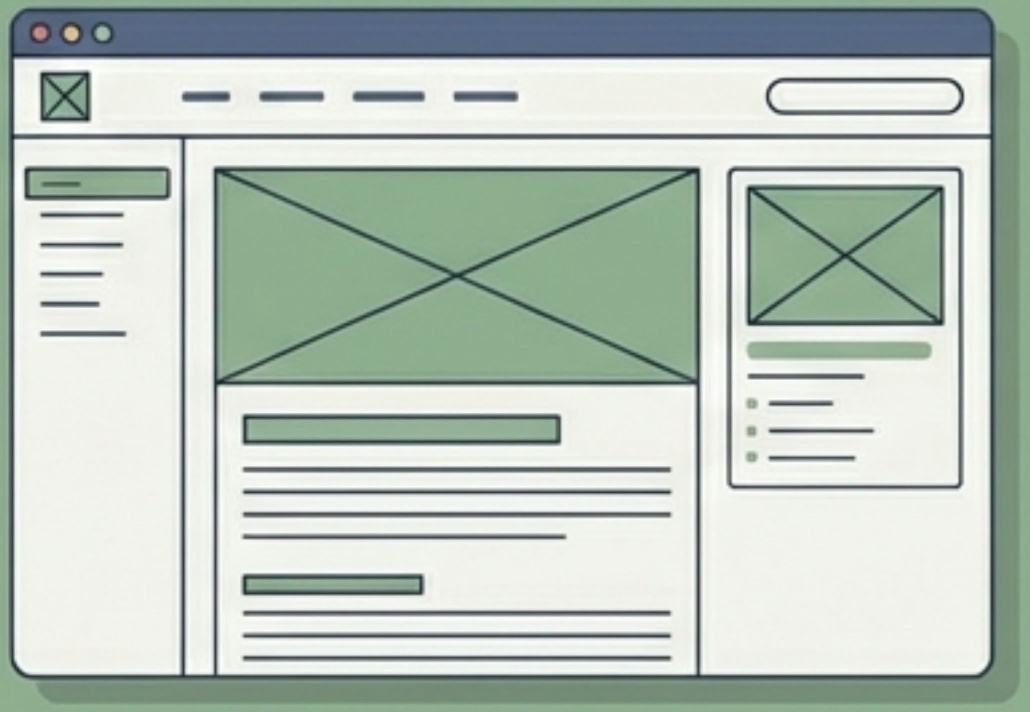
Agents building the cages for other agents.



Managed via 'Issues FS Pipe' (Filesystem-based tracking).

Roadmap & Deliverables

Public Documentation Hub



Concepts, Security Models,
Community Resources.
JetBrains Mono

Proof of Concept: Email MAL

Flagship Implementation.
Temporal & Content Filtering.
JetBrains Mono

Proof of Concept: Filesystem

Directory Scoping.

Proof of Concept: Calendar

Time-windowed Access.

Open Source & Community Driven.

From ‘Chatbot’ Toys to Agentic Work

We cannot trust agents with human-level permissions. The Model Abstraction Layer is the necessary bridge to move from experimental pilots to real-world deployment.

Implement MAL patterns before deploying agentic workflows.